



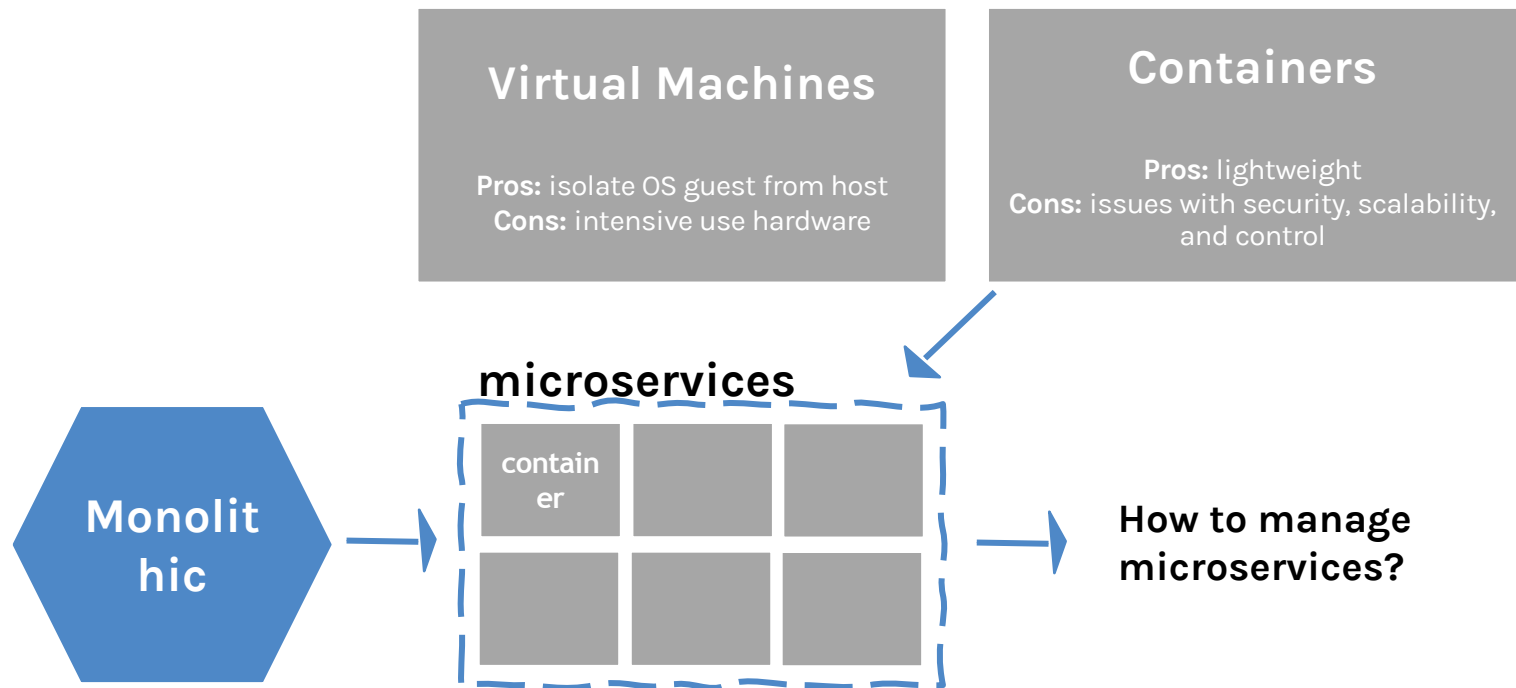
Wrocław University of Technology

# Kubernetes, Gitlab CI

Tomasz Walkowiak

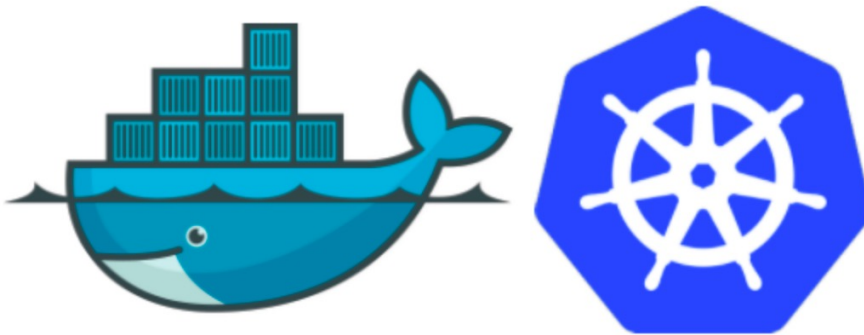


# RECAP





# Why container clusters?



**Oracle Developers** @OracleDevs · Jun 1  
#Kubernetes Community Engagement: Time to Roll!

Learn more about our commitment to k8s project at [blogs.oracle.com/developers/kub...](https://blogs.oracle.com/developers/kub...)



kubernetes

ORACLE



CoreOS

CoreOS, Inc., Timothy J Fontaine, Oracle and 4 others



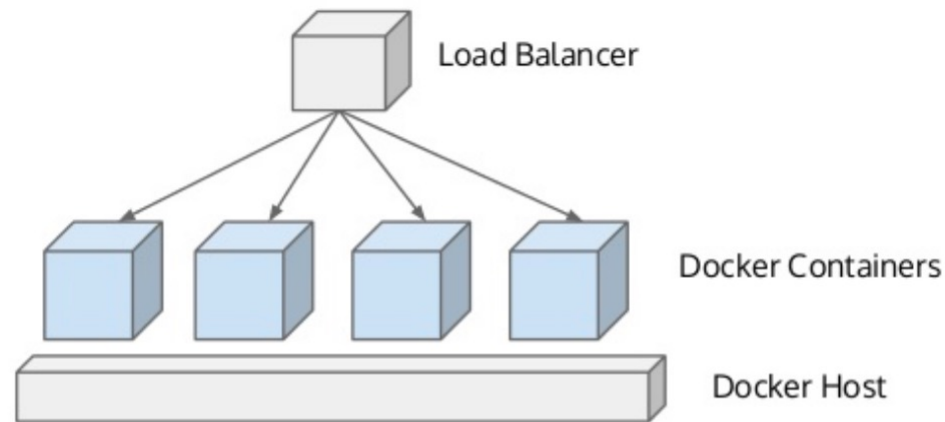
97



109



## Problems with standalone Docker



- Running a server cluster on a set of Docker containers, on a single Docker host is vulnerable to single point of failure!





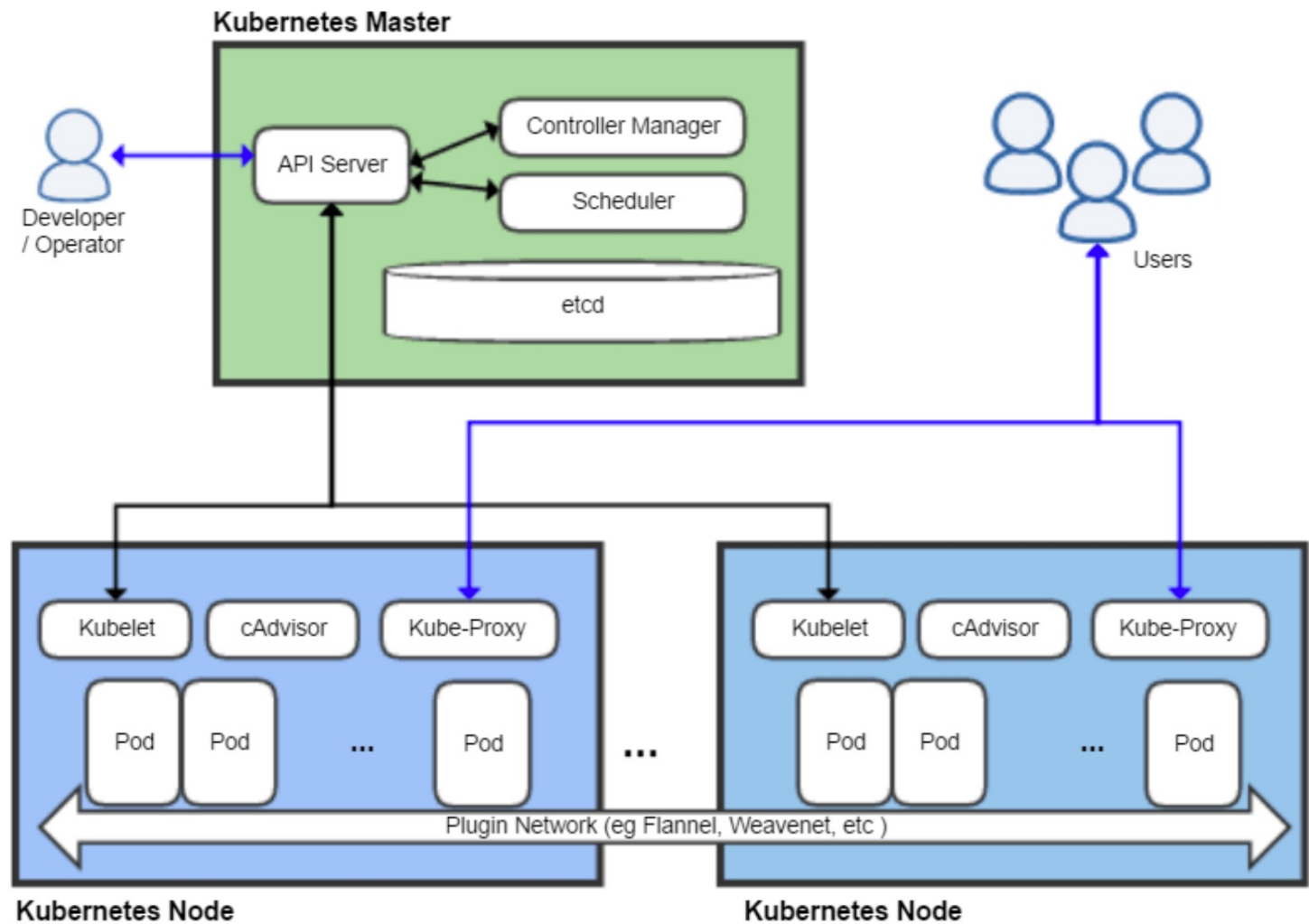
# Kubernetes (K8S)

- a portable, extensible open-source platform
- for managing containerized workloads and services,
- facilitates both declarative configuration and automation.
- It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

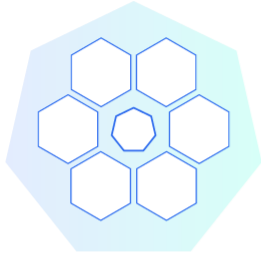


# Kubernetes (K8S)

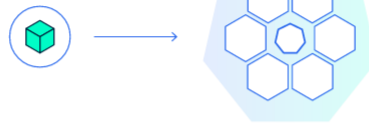
- K8s is an orchestration tool for **managing distributed services** or containerized applications across a distributed cluster of nodes.
- K8s itself follows a **client-server architecture with a master and worker nodes**. Core concepts in Kubernetes include pods, services (logical pods with a stable IP address) and deployments (a definition of the desired state for a pod or replica set).
- K8s **users define rules** for how container management should occur, and then K8s handles the rest!



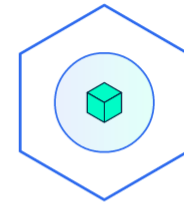
# Kubernetes Basics Modules



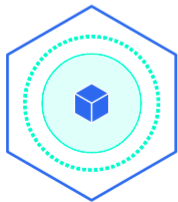
1. Create a Kubernetes cluster



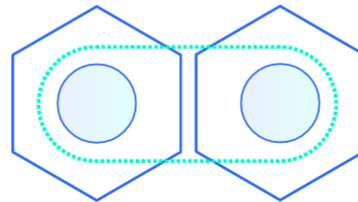
2. Deploy an app



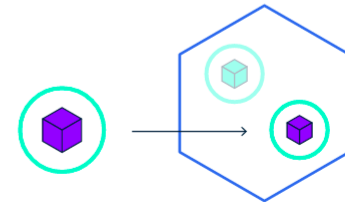
3. Explore your app



4. Expose your app publicly



5. Scale up your app



6. Update your app





- K8s is an orchestration tool for **managing distributed services** or containerized applications across a distributed cluster of nodes.
- K8s itself follows a **client-server architecture with a master and worker nodes**. Core concepts in Kubernetes include pods, services (logical pods with a stable IP address) and deployments (a definition of the desired state for a pod or replica set).
- K8s **users define rules** for how container management should occur, and then K8s handles the rest!

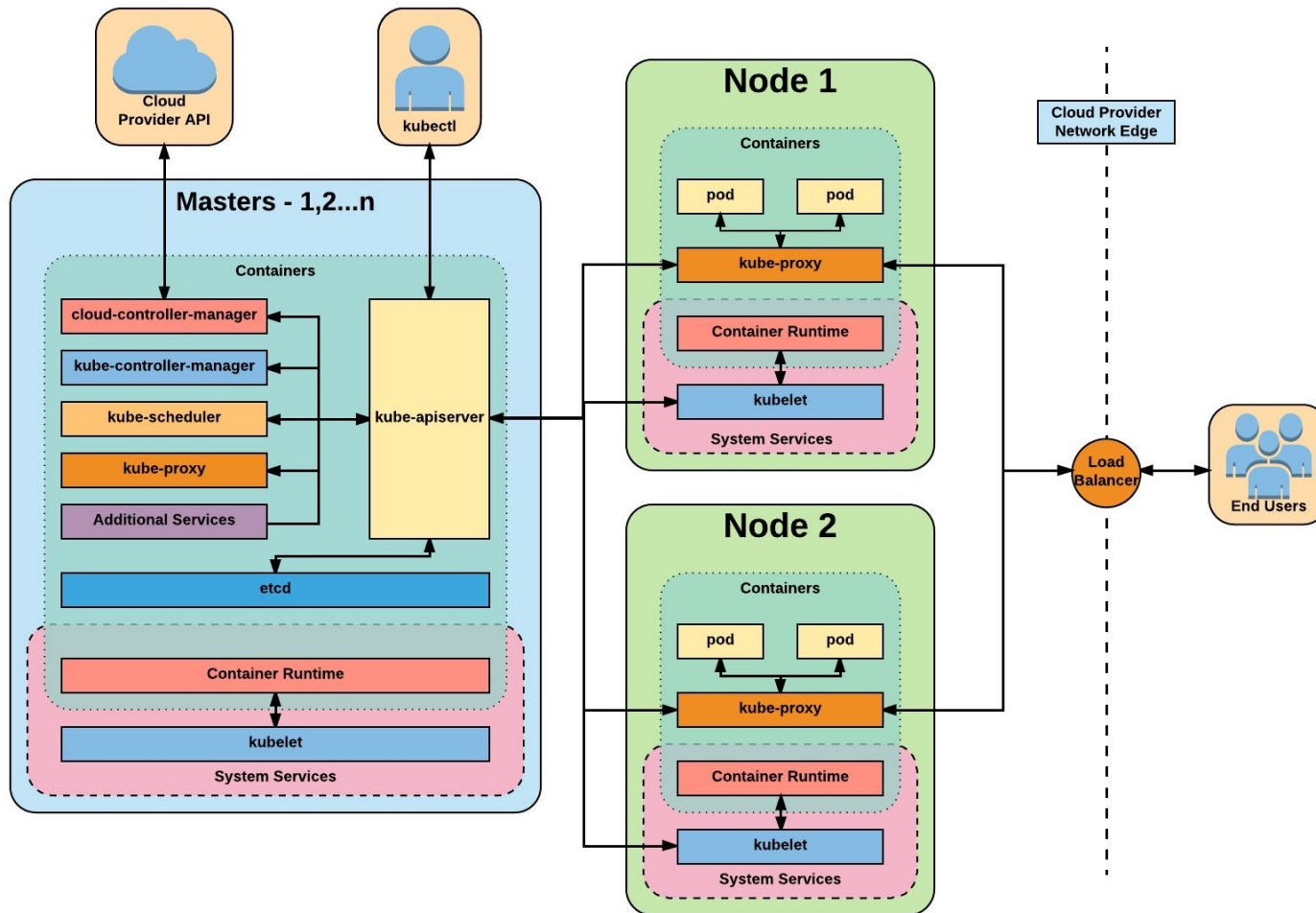


# Rancher

- Demo



# Architecture





# Kubernetes Concepts - Core

**Cluster** - A collection of hosts that aggregate their available resources including cpu, ram, disk, and their devices into a usable pool.

**Master** - The master(s) represent a collection of components that make up the control plane of Kubernetes. These components are responsible for all cluster decisions including both scheduling and responding to cluster events.

**Node** - A single host, physical or virtual capable of running pods. A node is managed by the master(s), and at a minimum runs both kubelet and kube-proxy to be considered part of the cluster.

**Namespace** - A logical cluster or environment. Primary method of dividing a cluster or scoping access.



## Concepts - Core (cont.)

**Label** - Key-value pairs that are used to **identify**, describe and group together related sets of objects. Labels have a strict syntax and available character set. \*

**Annotation** - Key-value pairs that contain **non-identifying** information or metadata. Annotations do not have the the syntax limitations as labels and can contain structured or unstructured data.

**Selector** - Selectors use labels to filter or select objects. Both equality-based (=, ==, !=) or simple key-value matching selectors are supported.

\* <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/#syntax-and-character-set>



# Kubectl

- Commands to complete the exercise:

```
$ kubectl create -f app-db-deploymnet.yaml
$ kubectl get deployment
$ kubectl get pods
$ kubectl get pods /
  -o=custom-
columns=NAME:.metadata.name,IP:.status.podIP
$ kubectl create -f app-server-deploymnet.yaml
$ kubectl expose deployment /
app-deployment --type=LoadBalancer --port=8080
$ kubectl get services
$ kubectl delete service app-deployment
$ kubectl delete deployment app-server-deployment
$ kubectl delete deployment app-db-deployment
```



# Labels, and Annotations, and Selectors

```
apiVersion: apps/v1beta2
kind: Deployment
metadata:
  name: nginx
  annotations:
    description: "nginx frontend"
  labels:
    app: nginx
    tier: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      tier: frontend
  template:
    metadata:
      labels:
        app: nginx
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Labels:

app: nginx tier:  
frontend

Annotations

description: "nginx frontend"

Selector: app: nginx

tier: frontend



## Concepts - Workloads

**Pod** - A pod is the smallest unit of work or management resource within Kubernetes. It is comprised of one or more containers that share their storage, network, and context (namespace, cgroups etc).

**ReplicationController** - Method of managing pod replicas and their lifecycle. Their scheduling, scaling, and deletion.

**ReplicaSet** - Next Generation ReplicationController. Supports set-based selectors.

**Deployment** - A declarative method of managing stateless Pods and ReplicaSets. Provides rollback functionality in addition to more granular update control mechanisms.





# Volumes

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - image: nginx:latest
    name: nginx
    volumeMounts:
    - mountPath: /usr/share/nginx/html
      name: www
  volumes:
  - name: www
    emptyDir: {}
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - image: nginx:latest
    name: nginx
    volumeMounts:
    - mountPath: /usr/share/nginx/html
      name: www
  volumes:
  - name: www
    awsElasticBlockStore:
      volumeID: <volume-id>
      fsType: ext4
```



# Persistent Volumes

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs
spec:
  capacity:
    storage: 500Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /data
    server: 10.255.100.10
```

- PVs are a cluster-wide resource
- Not directly consumable by a Pod
- PV Parameters:
  - Capacity
  - accessModes
    - ReadOnlyMany (ROX)
    - ReadWriteOnce (RWO)
    - ReadWriteMany (RWX)
  - persistentVolumeReclaimPolicy
    - Retain
    - Recycle
    - Delete
  - StorageClass



## Concepts - Configuration

- **ConfigMap** - Externalized data stored within kubernetes that can be referenced as a commandline argument, environment variable, or injected as a file into a volume mount. Ideal for separating containerized application from configuration.
- **Secret** - Functionally identical to ConfigMaps, but stored encoded as base64, and encrypted at rest (if configured).



# ConfigMaps and Secrets

- Can be used in Pod Config:
  - Injected as a file
  - Passed as an environment variable
  - Used as a container command (requires passing as env var)

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - image: nginx:latest
      name: nginx
      volumeMounts:
        - name: myConfigMap
          path: /etc/config
  volumes:
    - name: myConfigMap
      configMap:
        name: my-cm
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - image: nginx:latest
      name: nginx
      env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-cm
data:
  name: mydata.txt
  contents: |
    you can store
    multiline content
    and configfiles
```

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
data:
  username: aGVycGRlcnA=
  password: aW1tYWVnbXB1dGVy
```



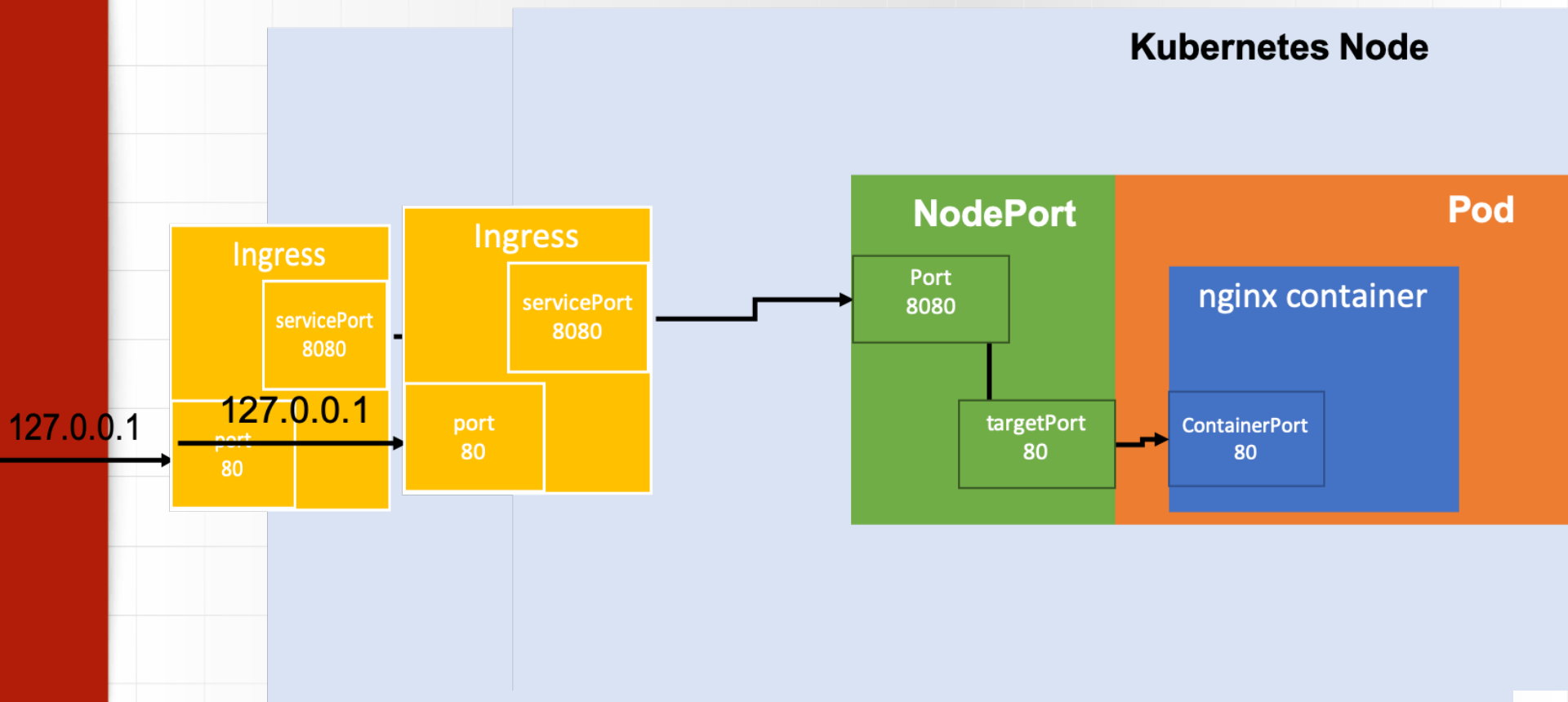
# Concepts - Network

**Service** - Services provide a method of exposing and consuming L4 Pod network accessible resources. They use label selectors to map groups of pods and ports to a cluster-unique virtual IP.

**Ingress** - An ingress controller is the primary method of exposing a cluster service (usually http) to the outside world. These are load balancers or routers that usually offer SSL termination, name-based virtual hosting etc.

# Network in k8s

- Internal network:
  - By build-in DNS (like in docker)
  - Namespaced – first looks in current namespace, postfix notation for other namespace (like pod\_name.**namespace**)
- External network:
  - Ingress (for example *nginx*)
    - Exposes HTTP and HTTPS routes from outside the cluster to services
  - NodePort
    - Exposes one port from specified range, service is reachable on NodeAddress:Nodeport
  - LoadBalancer
    - Traffic from the external load balancer is directed at the backend Pods. The cloud provider decides how it is load balanced



# Ingress paths

- Defines connection between service and router
- Can provide:
  - Externally-reachable URLs
  - Target rewrite
  - TLS
  - Loadbalancing (Avi Kubernetes Operator)
  - Valid Let's Encrypt certificate

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
    - hosts:
        - https-example.foo.com
      secretName: testsecret-tls
  rules:
    - host: https-example.foo.com
      http:
        paths:
          - path: /
            pathType: Prefix
        backend:
          service:
            name: service1
            port:
              number: 80
```



# Loadbalancing

- For a Service
  - Service type: LoadBalancer
  - Traffic from the external load balancer is directed at the backend Pods
  - Uses external loadbalancer
  - Operates at the L4 level
  - Supports traffic types like TCP, UDP
  - Assigned its own IP address
- With ingress
  - Operates at the L7 level
  - Services share one IP address

# Other load balancing

- PodAutoscaler
  - Automatically updates a workload resource to scale
  - If the load decreases, PodAutoscaler scale back down
  - Support resource metrics (like length of queue on Rabbitmq)
- HPA (HorizontalPodAutoscaler) - the response to increased load is to **deploy more Pods**
- VPA (VerticalPodAutoscaler) - the response to increased load is to **assigning more resources** (memory or CPU) to the existing Pods



# MiniKube

- To deploy your cluster you must install Kubernetes. In the exercise you are going to use minikube to deploy a cluster in local mode.

10.17.50.97

- After installing minikube, use *start* to begin your session creating a virtual machine, *stop* to interrupt it, and *delete* to remove the VM. Below are the commands to execute these tasks:

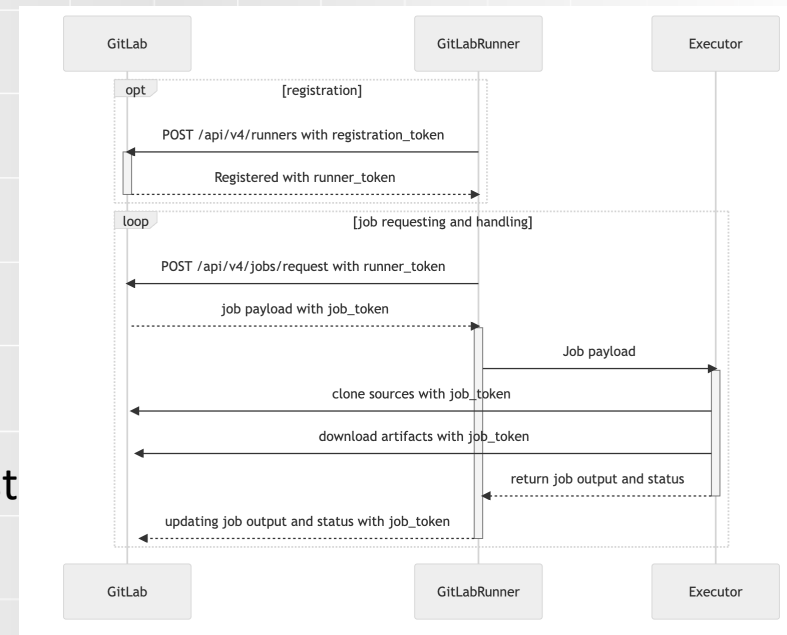
- `$ minikube start`
- `$ minikube stop`
- `$ minikube delete`

# CI

- Continuous integration (CI) is a software development practice where developers in a team integrate their work frequently.
- Developers usually integrate several times a day.
- Each integration is verified by an automated build: compile the code and also run automated tests
- Very often integrated with repositories

# GitLab-CI

- Fully integrated with gitlab platform
- Allow to configure pipelines consistings of jobs
- Pipeline triggered by push or scheduler
- Requires configuring runner
- Runner is docker container
- Can handle tasks like:
  - Checking code style
  - Running tests
  - Building container image
  - Store container image in gitlab regist
  - Publish docs in gitlab pages

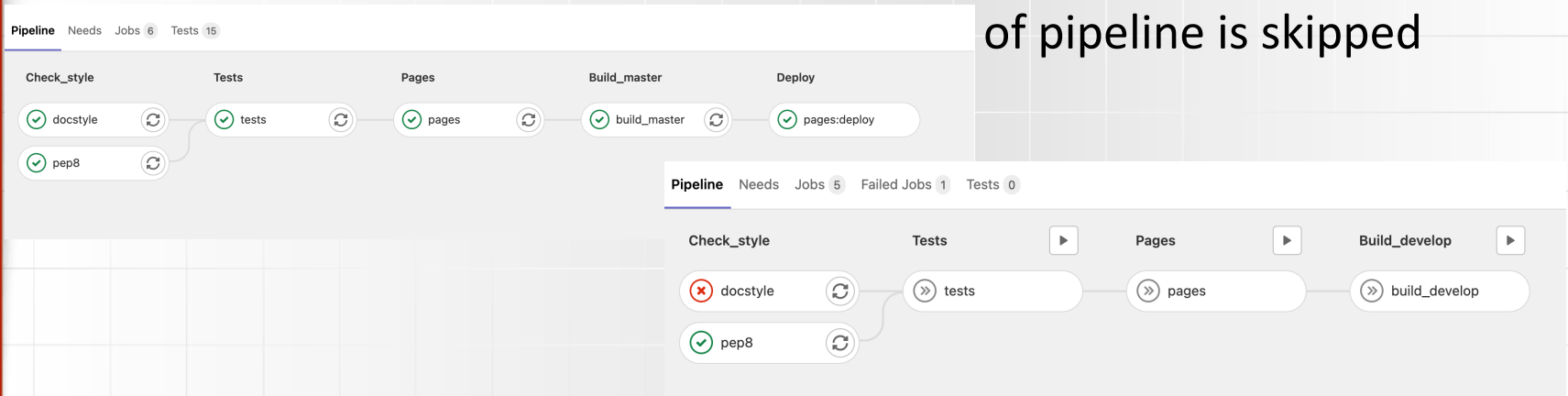


<https://docs.gitlab.com/runner/>

# Using GitLab-CI

- After runners configuration project pipeline is defined in ``.gitlab-ci.yml`` file
- Pipelines run on every repository push
- Crontab like scheduler is available
- Pipeline can use specified in group / project variables like login and passwords, which are masked in logs

of pipeline is skipped





# .gitlab-ci.yml

**image:** python:3.8 # pipeline docker image tag from dockerhub

**stages:** # list of jobs to run sequentially, jobs runs only if previous job succeded

- check\_style
- tests
- pages
- build\_master
- build\_develop

**pep8:** # job name

**stage:** check\_style # stage name

**before\_script:** # preparation steps

- pip install tox==2.9.1

**script:** # actual job

- tox -v -e pep8

**docstyle:**

**stage:** check\_style # using the same stage

**before\_script:**

- pip install tox==2.9.1

**script:**

- tox -v -e docstyle

**tests:**

**stage:** tests

**before\_script:**

- pip install tox==2.9.1

**script:**

- tox -v -e pytest

**artifacts:** # saving job artifacts, tests report

**paths:**

- htmlcov

**pages:** # this cannot be changed

**stage:** pages

**script:**

- mkdir -p public/coverage # gitlab pages are stored in public dire
- cp -r htmlcov/\* public/coverage/

**artifacts:**

**name:** coverage

**paths:**

- public

**build\_develop:**

**except:**

- master # job will run only if branch name is different than master

**stage:** build\_develop

**image:** docker:18.09.7 # docker builder image different than pipeli

**base image**

**services:**

- 'docker:18.09.7-dind'

**script:**

- docker build -t \$CI\_REGISTRY\_IMAGE:develop .
- docker login -u \$CI\_REGISTRY\_USER -p

\$CI\_REGISTRY\_PASSWORD \$CI\_REGISTRY

- docker push \$CI\_REGISTRY\_IMAGE:develop # publishing ima

gitlab registry

**build\_master:**

**stage:** build\_master

**image:** 'docker:18.09.7'

**only:**

- master # job will run only if branch is called master

**services:**

- 'docker:18.09.7-dind'

**script:**

- docker build -t \$CI\_REGISTRY\_IMAGE:latest .
- docker login -u \$CI\_REGISTRY\_USER -p

\$CI\_REGISTRY\_PASSWORD \$CI\_REGISTRY

- docker push \$CI\_REGISTRY\_IMAGE:latest